

llm_task2_1_Llama32_1B_seller_agentic_workflow20260314

March 14, 2026

0.1 Seller agentic workflow pipeline

This script implements an end-to-end pipeline that combines a Random Forest regression model with a local LLM (LLaMA-3.2-1B-Instruct via LM Studio) to support home sellers with pricing feedback and listing refinement.

Main components 1. Data and model loading 2. LLM connectivity and prompt construction - `call_llm(...)` sends a chat completion request to a local OpenAI-compatible API exposed by LM Studio, targeting the `llama-3.2-1b-instruct` model. - `build_seller_prompt(...)` creates a structured prompt that includes: * the Random Forest predicted price, * the four sentiment scores, * the original listing description. The prompt instructs the LLM to (a) explain the suggested price in 2–3 sentences and (b) produce an improved listing description in a fixed output format. 3. High-level agent function - `seller_assistant(...)` orchestrates the full workflow: * builds the feature vector for a given seller, * calls the Random Forest to predict a price, * calls the LLM to generate an explanation and an improved description conditioned on the predicted price and scores. - Returns a dictionary with: * `predicted_price`: numeric price estimate from the RF model, * `llm_seller_advice`: raw LLM text combining explanation and improved listing.

Intended usage In a thesis or prototype context, this pipeline demonstrates how a classic tabular ML model (Random Forest) can be combined with a local LLM to build an “agentic” assistant for home sellers: the RF model provides a grounded numeric estimate, while the LLM explains the estimate and rewrites the listing in a more attractive, score-aware way.

```
[ ]: import os
import pandas as pd
import numpy as np
import joblib
import requests
import textwrap

LM_STUDIO_URL = "http://localhost:1234/v1/chat/completions"
LM_MODEL_NAME = "llama-3.2-1b-instruct"

# For LLM: adapt imports depending on how you load LLaMA-3.2-1B-Instruct
# Example using transformers + GGUF/llama.cpp wrapper is omitted here;
# you will plug in your own client/model object where indicated.
```

```

# 1 Paths and data/model loading
current_dir = os.path.dirname(os.path.abspath("__file__")) if "__file__" in_
    ↪globals() else os.getcwd()
output_dir = os.path.join(current_dir, "output")

# 1.1) Load the enhanced dataset (with scores)
data_file = os.path.join(output_dir, "df_with_extracted_scores.csv")
df = pd.read_csv(data_file)
print("Loaded data:", data_file)
print("Columns:", df.columns.tolist())

# 1.2) Load the trained Random Forest model (with extracted scores)
model_path = os.path.join(output_dir, "rf_with_scores_model.pkl")
rf_model = joblib.load(model_path)
print("Loaded RF model from:", model_path)

# 1.3) Define feature columns used for training this model
num_cols = ["sizeSqFeetMax", "bedrooms", "bathrooms",
            ↪"luxury_score", "transport_score", "school_score",
            ↪"renovation_score"]

cat_cols = ["propertyType", "listingUpdateReason"]

# Build one-hot columns list from training data structure
# For correct prediction on new sellers, we will precompute the full set of_
    ↪dummies from df
df_cat = pd.get_dummies(df[cat_cols], prefix=cat_cols, drop_first=False)
all_cat_cols = df_cat.columns.tolist()

# 2 Helper: build feature vector for a new seller listing
def build_features_for_seller_input(
    seller_input: dict,
    luxury_score: int,
    transport_score: int,
    school_score: int,
    renovation_score: int
) -> pd.DataFrame:
    """
    seller_input keys expected:
    - sizeSqFeetMax (float or int)

```

```

- bedrooms (int)
- bathrooms (int)
- propertyType (string, e.g., 'House', 'Apartment', 'Terraced',
↳ 'Detached')
- listingUpdateReason (string, e.g., 'new', 'price_reduced')
The four *_score values should already be computed from a separate scoring
↳ step.
"""
# numeric part
row = {
    "sizeSqFeetMax": float(seller_input["sizeSqFeetMax"]),
    "bedrooms": float(seller_input["bedrooms"]),
    "bathrooms": float(seller_input["bathrooms"]),
    "luxury_score": float(luxury_score),
    "transport_score": float(transport_score),
    "school_score": float(school_score),
    "renovation_score": float(renovation_score),
}

# start DF with numeric columns
X_num = pd.DataFrame([row], columns=num_cols)

# categorical one-hot, aligned with training dummy columns
cat_base = {
    "propertyType": seller_input["propertyType"],
    "listingUpdateReason": seller_input["listingUpdateReason"],
}
df_temp_cat = pd.get_dummies(pd.DataFrame([cat_base]), prefix=cat_cols,
↳ drop_first=False)

# Ensure all training-time dummy columns exist
for col in all_cat_cols:
    if col not in df_temp_cat.columns:
        df_temp_cat[col] = 0

# Keep same column order as training
df_temp_cat = df_temp_cat[all_cat_cols]

# Combine numeric + categorical
X = pd.concat([X_num, df_temp_cat], axis=1)

```

```

return X

# 3 Helper: call LLaMA-3.2-1B-Instruct to improve description
def call_llm(prompt: str) -> str:
    """
    Call LLaMA-3.2-1B-Instruct via LM Studio's local OpenAI-compatible API.
    Make sure LM Studio server is running (Status: Running).
    """
    payload = {
        "model": LM_MODEL_NAME,
        "messages": [
            {"role": "system", "content": "You are a helpful real-estate_↵
↵assistant."},
            {"role": "user", "content": prompt}
        ],
        "temperature": 0.3,
        "max_tokens": 512,
    }

    try:
        resp = requests.post(LM_STUDIO_URL, json=payload, timeout=120)
        resp.raise_for_status()
        data = resp.json()
        # LM Studio returns OpenAI-style 'choices'
        return data["choices"][0]["message"]["content"]
    except Exception as e:
        print("Error calling LLM:", e)
        return "Error: LLM call failed."

def build_seller_prompt(original_description: str,
                        luxury_score: int,
                        transport_score: int,
                        school_score: int,
                        renovation_score: int,
                        predicted_price: float) -> str:
    instructions = f"""
You are an assistant helping a home seller improve their listing.

1) The Random Forest model predicted a price of approximately &{predicted_price:
↵,.0f}.

2) The following sentiment scores were extracted from the current description:
- luxury_score (0-2): {luxury_score}
- transport_score (0/1): {transport_score}
- school_score (0/1): {school_score}
- renovation_score (0/1): {renovation_score}

```

Task:

- First, write ONLY a short explanation of the suggested price in 2-3 sentences, focusing on size, bedrooms, bathrooms, and sentiment scores.
- Then write ONLY ONE improved listing description.
- In the improved description:
 - Emphasise genuine luxury features if they exist.
 - Clearly mention good transport links and school quality if present.
 - Highlight any renovations or condition improvements.
- Keep the style natural and suitable for a real estate listing.
- Do not invent facts: only emphasise or rephrase what a typical property with these scores would have.
- Return your answer in exactly this format:

Explanation:

<one short explanation paragraph>

Current listing description:

<the original description, possibly lightly cleaned up but not changed in meaning>

Improved description:

<one improved listing description paragraph>

"""

```
prompt = f"""{instructions.strip()}
```

Current listing description:

```
\\"\"\"{original_description.strip()}\\\"\"\"
```

"""

```
return prompt
```

```
def improve_listing_with_llm(original_description: str,
                             luxury_score: int,
                             transport_score: int,
                             school_score: int,
                             renovation_score: int,
                             predicted_price: float) -> str:
    prompt = build_seller_prompt(
        original_description,
        luxury_score,
        transport_score,
        school_score,
        renovation_score,
        predicted_price
    )
    return call_llm(prompt)
```

```

# 4 End-to-end seller agent function
def seller_assistant(
    seller_input: dict,
    scores: dict,
    original_description: str
) -> dict:
    """
    High-level agentic workflow for a seller.

    seller_input:
        - sizeSqFeetMax
        - bedrooms
        - bathrooms
        - propertyType
        - listingUpdateReason

    scores:
        - luxury_score
        - transport_score
        - school_score
        - renovation_score

    original_description:
        - The seller's current listing description (string).

    Returns:
        - dict with predicted_price, explanation+improved_description (from LLM).
    """
    # 1) Build features
    X = build_features_for_seller_input(
        seller_input=seller_input,
        luxury_score=scores["luxury_score"],
        transport_score=scores["transport_score"],
        school_score=scores["school_score"],
        renovation_score=scores["renovation_score"],
    )

    # 2) Predict price with RF model
    predicted_price = float(rf_model.predict(X)[0])

```

```

# 3) Ask LLM to explain price and improve description
llm_output = improve_listing_with_llm(
    original_description=original_description,
    luxury_score=scores["luxury_score"],
    transport_score=scores["transport_score"],
    school_score=scores["school_score"],
    renovation_score=scores["renovation_score"],
    predicted_price=predicted_price
)

return {
    "predicted_price": predicted_price,
    "llm_seller_advice": llm_output
}

# 5 Example usage (once you plug in the LLM)
example_seller_input = {
    "sizeSqFeetMax": 4000,
    "bedrooms": 4,
    "bathrooms": 4,
    "propertyType": "House",
    "listingUpdateReason": "new",
}

example_scores = {
    "luxury_score": 2,
    "transport_score": 1,
    "school_score": 1,
    "renovation_score": 1,
}

example_description = """
A beautifully presented family house with spacious rooms, a modern kitchen,
and a landscaped garden. Located close to transport links and good local
↪schools.
"""

result = seller_assistant(example_seller_input, example_scores,
↪example_description)

```

Loaded data:

c:\Users\Admin\Python\S8_Thesis\llm\output\df_with_extracted_scores.csv
Columns: ['title', 'propertyType', 'sizeSqFeetMax', 'bedrooms', 'bathrooms',
'testingUpdateReason', 'price', 'Date', 'listingDescription', 'luxury_score',
'transport_score', 'school_score', 'renovation_score']

Loaded RF model from:

c:\Users\Admin\Python\S8_Thesis\llm\output\rf_with_scores_model.pkl

```
[ ]: # 6 Example usage (UI-style display for thesis)
from textwrap import fill

def display_seller_session(seller_input, original_description, result):
    """
    Simple UI-style text display for Jupyter / thesis screenshots.
    """
    print("=" * 80)
    print("SELLER ASSISTANT - INPUT".center(80))
    print("=" * 80)

    # Seller basic info
    print("\n[Seller listing metadata]")
    print(f"  Property type      : {seller_input['propertyType']}")
    print(f"  Size (sq ft, max)   : {seller_input['sizeSqFeetMax']}")
    print(f"  Bedrooms            : {seller_input['bedrooms']}")
    print(f"  Bathrooms           : {seller_input['bathrooms']}")
    print(f"  Listing update reason: {seller_input['listingUpdateReason']}")

    # Original description
    print("\n[Original listing description]")
    print(fill(original_description.strip(), width=80))

    print("\n" + "-" * 80)
    print("MODEL OUTPUT".center(80))
    print("-" * 80)

    # Predicted price
    print(f"\nPredicted price (Random Forest): &{result['predicted_price']:,}
    ↪0f}")

    # Raw LLM output
    llm_text = result["llm_seller_advice"]

    # --- Parse Explanation / Current / Improved for nicer display ---
    parts = llm_text.rsplit("Improved description:", 1)
    before_last_improved = parts[0]
    last_improved_block = parts[1] if len(parts) == 2 else ""

    exp_parts = before_last_improved.split("Explanation:", 1)
    explanation_text = exp_parts[1].strip() if len(exp_parts) == 2 else
    ↪before_last_improved.strip()

    # Try to extract "Current listing description" from the LLM output if
    ↪present,
```



```

# but keep it optional. If not found, we just show the original again.
curr_parts = explanation_text.split("Current listing description:", 1)
if len(curr_parts) == 2:
    explanation_only = curr_parts[0].strip()
    llm_current_desc = curr_parts[1].strip()
else:
    explanation_only = explanation_text
    llm_current_desc = original_description.strip()

# Clean the improved description:
# remove any trailing "Current listing description: ..." if the model
# accidentally appended it.
if "Current listing description:" in last_improved_block:
    improved_only = last_improved_block.split("Current listing description:
↵")[0].strip()
else:
    improved_only = last_improved_block.strip()

# Remove any surrounding quotes
improved_only = improved_only.strip().strip('\"')

# Pretty wrapping
wrapped_explanation = fill(explanation_only, width=80)
wrapped_current = fill(llm_current_desc, width=80)
wrapped_improved = fill(improved_only, width=80)

print("\n[Explanation of suggested price]")
print(wrapped_explanation)

print("\n[Current listing description]")
print(wrapped_current)

print("\n[Improved listing description]")
print(wrapped_improved)

print("\n" + "=" * 80)

# Run one example seller session
example_seller_input = {
    "sizeSqFeetMax": 4000,
    "bedrooms": 4,
    "bathrooms": 4,
    "propertyType": "House",
    "listingUpdateReason": "new",
}

```

```

example_description = """
A beautifully presented family house with spacious rooms, a modern kitchen,
and a landscaped garden. Located close to transport links and good local
↳schools.
"""

result = seller_assistant(exampleSellerInput, example_scores,
↳example_description)

# Display in simple UI-like format
displaySellerSession(exampleSellerInput, example_description, result)

```

SELLER ASSISTANT - INPUT

[Seller listing metadata]

```

Property type      : House
Size (sq ft, max)  : 4000
Bedrooms           : 4
Bathrooms          : 4
Listing update reason: new

```

[Original listing description]

A beautifully presented family house with spacious rooms, a modern kitchen, and a landscaped garden. Located close to transport links and good local schools.

MODEL OUTPUT

Predicted price (Random Forest): £21,189,655

[Explanation of suggested price]

** The Random Forest model predicted a price of approximately £21,189,655 due to its high luxury score (2) and strong sentiment scores indicating excellent transport links and school quality.

[Current listing description]

A beautifully presented family house with spacious rooms, a modern kitchen, and a landscaped garden. Located close to transport links and good local schools.

**

[Improved listing description]

** This luxurious family home boasts an impressive 5-bedroom layout, a sleek modern kitchen, and expansive outdoor spaces perfect for entertaining. Situated in a highly sought-after area with excellent transport links and top-rated local

schools, this property offers the ultimate in comfort and convenience. With its high-end finishes and premium features, it's the epitome of luxury living.

=====